

Homework #1 Submission

✓ **1. Setup proof.** Install a JDK (Java 21 or 25), IntelliJ IDEA, and Git. Create the `phase1-java` repo and commit `Hello.java`. Send me the output of `java -version` and your repo link.

```
java --version
openjdk 21.0.10 2026-01-20 LTS
OpenJDK Runtime Environment (build 21.0.10+10-LTS)
OpenJDK 64-Bit Server VM (build 21.0.10+10-LTS, mixed mode, sharing)

repo link: https://github.com/deskthomas/phase1-java.git
```

✓ **2. Deductible payout.** Read a claim amount and a deductible, then print the payout. Then test it with a deductible larger than the claim. In a comment, explain what goes wrong and how you'd want it to behave. You'll fix it in Week 2.

```
        System.out.println("Please enter the amount o
f your insurance claim: ");
        double claimAmount = in.nextDouble();
        System.out.println("Please enter the amount of your d
eductible: ");
        double deductibleAmount = in.nextDouble();

        double payout = claimAmount - deductibleAmount;

        if (payout <= 0) {
            System.out.println("Your deductible of $" + deduc
tibleAmount + " is greater than or equal to the claim amount
$" + claimAmount);
            System.out.println("There is no payout.");
        } else {
            System.out.println("Your payout is: $" + payout);
        }
    }
}
```

Test Inputs

```
Please enter the amount of your insurance claim:
1000
Please enter the amount of your deductible:
500
Your payout is: $500.0
Please enter the amount of miles being claimed for reimbursement:
240
Please enter the mileage rate in decimal form:
0.74
Your reimbursement is: $177.6
Your reimbursement rounded to whole dollars is: $177
```

✓ **3. Mileage reimbursement.** Read miles driven and a rate per mile, then print the reimbursement. Then print it again rounded to whole dollars, using only casting and arithmetic. Hint: think about what `(int)` does to `x + 0.5`:

```
System.out.println("Miles Driven: ");
double milesDriven = in.nextDouble();
System.out.println("Rate: ");
double rate = in.nextDouble();

double mileageReimbursement = milesDriven * rate;

System.out.println("Your reimbursement is: $" + mileageReimbursement);
System.out.println("Your reimbursement rounded to whole dollars is: $" + (int) mileageReimbursement);
```

✓ **4. Type experiments.** In `predictions.txt`, write what you think each of these prints *before* running it. Then run them all and explain every one you got wrong.

- `7 / 2`, `7 / 2.0`, `(int) 3.99`
- `"1" + 2 + 3`, `1 + 2 + "3"`
- `10 % 3`, `-7 / 2`

Experiments

7/2:

Prediction: Prints 3

Actual Output: 3

7/2.0:

Prediction: Prints 3.5

Actual Output: 3.5

(int)3.99:

Prediction: Prints 3

Actual Output: 3

"1" + 2 + 3:

Prediction: Prints 15

Actual Output: 123

Why I was wrong: Reading left to right, the systems prints the string 1, then prints 2 and 3 sequentially instead of printing the arithmetic value

1 + 2 + "3":

Prediction: Prints 33

Actual Output: 33

10 % 3:

Prediction: Prints 1

Actual Output: 1

-7 / 2:

Prediction: Prints -3

Actual Output: -3

Self-check before you submit. Can you explain the difference between `javac` and `java`? Why does `"1" + 2 + 3` differ from `1 + 2 + "3"`?

javac compiles the source code while **java** executes the compiled code. `"1" + 2 + 3` first prints the string then performs integer concatenation for the remaining output. `1 + 2 + "3"` performs the arithmetic first then performs string concatenation for the remaining output. This indicates that when the lines are being executed, the compiler must decide between performing concatenation or calculation, with its decision being based on whether the first values being read from left to right is a string or an integer.